Garbled Circuits

Calcular juntos sin revelar nada

Alberto Lopez Barragan

Tecnológico de Monterrey

La pregunta imposible

¿Pueden dos personas que **no confían entre sí**

calcular algo juntas sobre sus datos secretos

sin revelárselos y **sin un árbitro**?

La pregunta imposible

¿Pueden dos personas que **no confían entre sí**

calcular algo juntas sobre sus datos secretos

sin revelárselos y **sin un árbitro**?

La intuición dice que no.
La criptografía dice que sí.

El problema de los millonarios (Yao, 1982)

El escenario

- Alicia tiene una fortuna x.
- Bob tiene una fortuna y.
- Quieren saber **quién es más rico**.
- Ninguno quiere decir cuánto tiene.

El problema de los millonarios (Yao, 1982)

El escenario

- Alicia tiene una fortuna x.
- Bob tiene una fortuna y.
- Quieren saber **quién es más rico**.
- Ninguno quiere decir cuánto tiene.

La solución obvia... y su costo

Contratar a un notario de confianza.

- ¿Y si el notario miente?
- ¿Y si lo hackean?
- ¿Y si vende los datos?

El problema de los millonarios (Yao, 1982)

El escenario

- Alicia tiene una fortuna x.
- Bob tiene una fortuna y.
- Quieren saber **quién es más rico**.
- Ninguno quiere decir cuánto tiene.

La solución obvia... y su costo

Contratar a un notario de confianza.

- ¿Y si el notario miente?
- ¿Y si lo hackean?
- ¿Y si vende los datos?

Lo que queremos

Eliminar al notario. Reemplazar la **confianza** por **matemáticas**.

Esto no es un juego de mesa

Medicina

Dos hospitales quieren saber cuántos pacientes comparten, sin intercambiar expedientes.

Finanzas

Dos bancos rivales quieren detectar fraude común, sin abrirse los libros.

Inteligencia artificial

Un cliente quiere la predicción de un modelo sin revelar sus datos; la empresa no quiere revelar el modelo.

Genómica

Comparar ADN para pruebas de parentesco, sin exponer el genoma.

Esto no es un juego de mesa

Medicina

Dos hospitales quieren saber cuántos pacientes comparten, sin intercambiar expedientes.

Inteligencia artificial

Un cliente quiere la predicción de un modelo sin revelar sus datos; la empresa no quiere revelar el modelo.

Finanzas

Dos bancos rivales quieren detectar fraude común, sin abrirse los libros.

Genómica

Comparar ADN para pruebas de parentesco, sin exponer el genoma.

Todos son el mismo problema: calcular $f(x, y)$ revelando únicamente el resultado.

¿Qué son los garbled circuits?

Son **la solución** al problema que acabamos de plantear.

Definición

Una **técnica criptográfica** que permite a dos partes calcular juntas una función sobre sus datos privados, sin revelárselos y sin un tercero de confianza. Cada una aprende **únicamente el resultado**.

¿Qué son los garbled circuits?

Son **la solución** al problema que acabamos de plantear.

Definición

Una **técnica criptográfica** que permite a dos partes calcular juntas una función sobre sus datos privados, sin revelárselos y sin un tercero de confianza. Cada una aprende **únicamente el resultado**.

"Circuit"	la función se escribe como circuito de puertas lógicas
"Garbled"	*distorsionado*: por los cables ya no viajan bits, sino **etiquetas aleatorias** que ocultan su valor

Las tres promesas que cumple

1. Exactitud

El resultado es **el correcto**: exactamente el mismo que daría calcular $f(x, y)$ normalmente.

2. Privacidad

Ninguna parte aprende **nada** sobre la entrada de la otra, más allá de lo que el resultado implique.

3. Sin terceros

No hay árbitro, ni servidor de confianza, ni notario. Solo las dos partes.

Las tres promesas que cumple

1. Exactitud

El resultado es **el correcto**: exactamente el mismo que daría calcular $f(x, y)$ normalmente.

2. Privacidad

Ninguna parte aprende **nada** sobre la entrada de la otra, más allá de lo que el resultado implique.

3. Sin terceros

No hay árbitro, ni servidor de confianza, ni notario. Solo las dos partes.

Lo que NO son

- No son cifrado normal (eso protege datos guardados o en tránsito, no permite *calcular* sobre ellos).
- No son blockchain, ni hardware seguro, ni anonimización.
- No ocultan la **función**: ambas partes saben qué se calcula. Lo que se oculta son las **entradas**.

¿Cuándo se usan y cuándo no?

Encajan bien cuando...

- Son **exactamente dos** partes.
- La función tiene muchas **comparaciones** (mayor que, máximo, igualdad).
- La red tiene **latencia alta**: el protocolo usa un número *constante* de rondas.
- Se necesita privacidad **demostrable**, no basada en confiar en alguien.

No encajan cuando...

- Hay **tres o más** partes (ahí se usan otras técnicas).
- La función es sobre todo **aritmética** (multiplicaciones grandes).
- Hay que reutilizar el mismo cálculo muchas veces: son de **un solo uso**.
- El ancho de banda es crítico y el circuito es enorme.

Quién los inventó: Andrew Yao



- Científico de la computación, nacido en Shanghái en 1946.

Quién los inventó: Andrew Yao



- Científico de la computación, nacido en Shanghái en 1946.
- **1982:** plantea el problema de los millonarios en un artículo sobre computación segura.

Quién los inventó: Andrew Yao



- Científico de la computación, nacido en Shanghái en 1946.
- **1982:** plantea el problema de los millonarios en un artículo sobre computación segura.
- **1986:** presenta la construcción de los circuitos cifrados. Curiosamente, **nunca la publicó formalmente**; circuló como folclore hasta que otros la escribieron y demostraron.

Quién los inventó: Andrew Yao



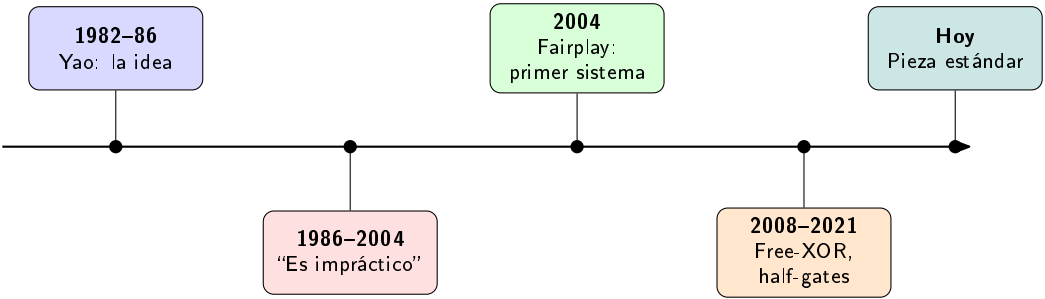
- Científico de la computación, nacido en Shanghái en 1946.
- **1982:** plantea el problema de los millonarios en un artículo sobre computación segura.
- **1986:** presenta la construcción de los circuitos cifrados. Curiosamente, **nunca la publicó formalmente**; circuló como folclore hasta que otros la escribieron y demostraron.
- **2000:** recibe el **Premio Turing** (el "Nobel de la computación").

Quién los inventó: Andrew Yao

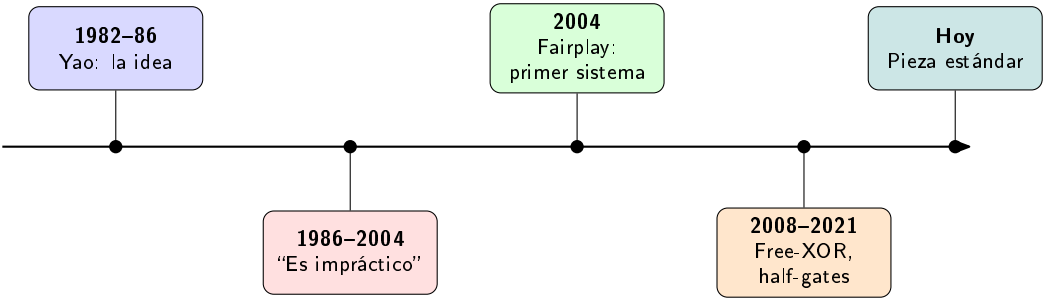


- Científico de la computación, nacido en Shanghái en 1946.
- **1982:** plantea el problema de los millonarios en un artículo sobre computación segura.
- **1986:** presenta la construcción de los circuitos cifrados. Curiosamente, **nunca la publicó formalmente**; circuló como folclore hasta que otros la escribieron y demostraron.
- **2000:** recibe el **Premio Turing** (el "Nobel de la computación").
- Hoy dirige investigación en la Universidad Tsinghua, China.

De curiosidad teórica a tecnología real



De curiosidad teórica a tecnología real



El salto de rendimiento

Durante casi 20 años se consideró una idea bella pero inútil en la práctica. Hoy, un sistema como **JustGarble** cifra una puerta en unos **7 nanosegundos**: millones de veces más rápido que las primeras implementaciones.

Fairplay (2004): el primer sistema que lo ejecutó

Quiénes

Malkhi, Nisan, Pinkas y Sella
Universidad Hebrea de Jerusalén

Qué construyeron

- Un **lenguaje** (SFDL) para describir la función con variables, `if` y operaciones normales.
- Un **compilador** que la traduce automáticamente a circuito booleano.
- El **sistema de red** que ejecuta garbling, OT y evaluación entre dos máquinas.

Fairplay (2004): el primer sistema que lo ejecutó

Quiénes

Malkhi, Nisan, Pinkas y Sella
Universidad Hebrea de Jerusalén

Qué construyeron

- Un **lenguaje** (SFDL) para describir la función con variables, `if` y operaciones normales.
- Un **compilador** que la traduce automáticamente a circuito booleano.
- El **sistema de red** que ejecuta garbling, OT y evaluación entre dos máquinas.

Por qué importó

Antes de Fairplay, los circuitos se diseñaban **a mano**. Ellos automatizaron el camino "función → circuito → protocolo".

Fue lento para los estándares de hoy (segundos donde ahora hay milisegundos), pero demostró lo importante: **esto se puede correr de verdad**.

Después de Fairplay se abrió la carrera de optimizaciones.

Los dos roles

Garbler (Alicia) — "la que cifra"

- Convierte la función en circuito.
- Genera todas las etiquetas.
- **Cifra** el circuito completo.
- Se lo manda a Bob.
- **Nunca ve la evaluación.**

Evaluator (Bob) — "el que evalúa"

- Recibe el circuito cifrado.
- Consigue las etiquetas de sus bits.
- Lo **evalúa a ciegas**.
- No entiende nada de lo que pasa dentro.
- Obtiene el resultado final.

Los dos roles

Garbler (Alicia) — "la que cifra"

- Convierte la función en circuito.
- Genera todas las etiquetas.
- **Cifra** el circuito completo.
- Se lo manda a Bob.
- **Nunca ve la evaluación.**

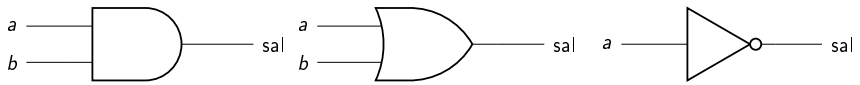
Evaluator (Bob) — "el que evalúa"

- Recibe el circuito cifrado.
- Consigue las etiquetas de sus bits.
- Lo **evalúa a ciegas**.
- No entiende nada de lo que pasa dentro.
- Obtiene el resultado final.

Los roles son **asimétricos**: por eso el protocolo clásico es estrictamente para **dos** partes.

¿Qué es una puerta lógica (*gate*)?

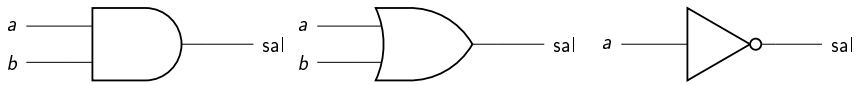
Una computadora, por dentro, solo maneja **bits**: 0 y 1. Una **puerta lógica** es una operación diminuta que recibe uno o dos bits y produce uno de salida, siguiendo una regla fija.



AND (“y”) **OR** (“o”) **NOT** (“no”)

¿Qué es una puerta lógica (*gate*)?

Una computadora, por dentro, solo maneja **bits**: 0 y 1. Una **puerta lógica** es una operación diminuta que recibe uno o dos bits y produce uno de salida, siguiendo una regla fija.



AND ("y") **OR** ("o") **NOT** ("no")

Con estas tres se puede construir **cualquier** cálculo que exista. Son el alfabeto de la computación.

Qué hace cada puerta, en concreto

AND			OR			NOT	
a	b	sal	a	b	sal	a	sal
0	0	0	0	0	0	0	**1**
0	1	0	0	1	**1**	1	0
1	0	0	1	0	**1**		
1	1	**1**	1	1	**1**		

Qué hace cada puerta, en concreto

AND			OR			NOT	
a	*b*	sal	*a*	*b*	sal	*a*	sal
0	0	0	0	0	0	0	**1**
0	1	0	0	1	**1**	1	0
1	0	0	1	0	**1**		
1	1	**1**	1	1	**1**		

Léelas como frases

AND: "sale 1 **solo si** ambas son 1".
OR: "sale 1 **si alguna** es 1".
NOT: "da lo contrario".

Dos más que usaremos

XOR: sale 1 si son *distintas*.
XNOR: sale 1 si son *iguales*.
(XNOR es el NOT de XOR.)

De una frase a una expresión booleana

Una **expresión booleana** es la fórmula que describe qué queremos calcular. Ejemplo cotidiano:

"La alarma suena si la puerta está abierta ***y*** *el sistema está activado"*

De una frase a una expresión booleana

Una **expresión booleana** es la fórmula que describe qué queremos calcular. Ejemplo cotidiano:

*"La alarma suena si la puerta está abierta **y** el sistema está activado"*

$$\text{alarma} = \text{puerta} \wedge \text{activado}$$

De una frase a una expresión booleana

Una **expresión booleana** es la fórmula que describe qué queremos calcular. Ejemplo cotidiano:

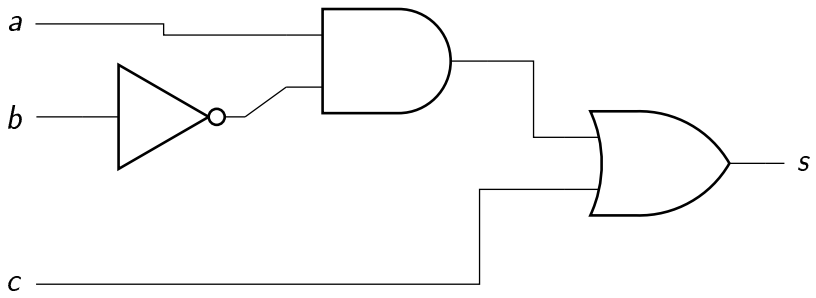
*"La alarma suena si la puerta está abierta **y** el sistema está activado"*

$$\text{alarma} = \text{puerta} \wedge \text{activado}$$

Símbolo	Significado
$\wedge$	AND ("y")
$\vee$	OR ("o")
$\neg$	NOT ("no")
$\odot$	XNOR ("son iguales")

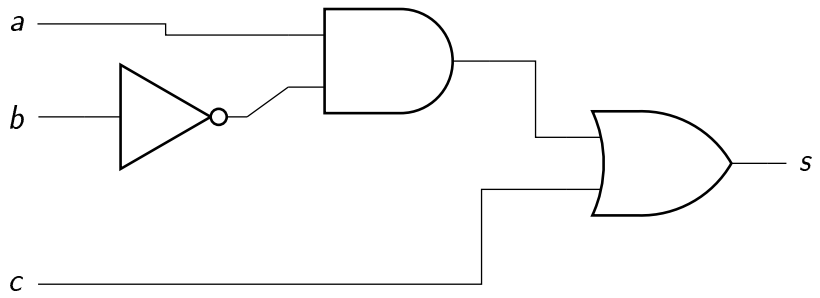
Una expresión booleana ES un circuito

Toda expresión se traduce, símbolo por símbolo, a puertas conectadas. Ejemplo:
$s = (a \wedge \neg b) \vee c$



Una expresión booleana ES un circuito

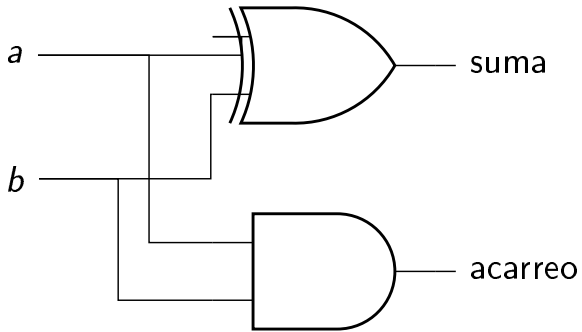
Toda expresión se traduce, símbolo por símbolo, a puertas conectadas. Ejemplo:
$s = (a \wedge \neg b) \vee c$



Se lee de izquierda a derecha: primero se niega b, luego se combina con a mediante AND, y ese resultado entra al OR junto con c.

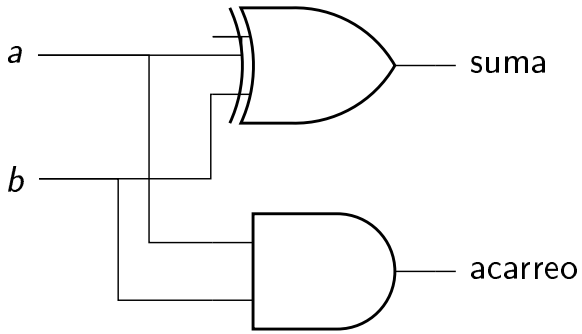
Ejemplo real: sumar dos bits con puertas

Sumar $1 + 1$ en binario da 10: un **resultado** (0) y un **acarreo** (1). Se construye con solo dos puertas:



Ejemplo real: sumar dos bits con puertas

Sumar $1 + 1$ en binario da 10: un **resultado** (0) y un **acarreo** (1). Se construye con solo dos puertas:

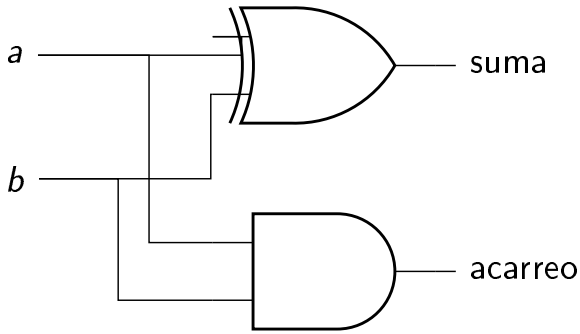


a	b	**suma** (XOR)	**acarreo** (AND)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	**1**

$1 + 1 = 10$: sale 0 y "me llevo una".

Ejemplo real: sumar dos bits con puertas

Sumar $1 + 1$ en binario da 10: un **resultado** (0) y un **acarreo** (1). Se construye con solo dos puertas:



a	b	**suma** (XOR)	**acarreo** (AND)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	**1**

$1 + 1 = 10$: sale 0 y "me llevo una".

Encadenando 64 de estos bloques se suma como lo hace tu computadora. **Todo cálculo es un circuito de puertas.**

¿Qué es la criptografía simétrica?

Simétrica

La misma llave cifra y descifra.

- Ejemplo: **AES**.
- **Muy rápida** (hay instrucciones de hardware).
- Problema: hay que compartir la llave de algún modo.

Asimétrica (clave pública)

Llaves **distintas** para cifrar y descifrar.

- Ejemplo: **RSA**, Diffie-Hellman.
- **Lenta** (matemáticas pesadas).
- Ventaja: no hace falta compartir un secreto previo.

¿Qué es la criptografía simétrica?

Simétrica

La misma llave cifra y descifra.

- Ejemplo: **AES**.
- **Muy rápida** (hay instrucciones de hardware).
- Problema: hay que compartir la llave de algún modo.

Asimétrica (clave pública)

Llaves **distintas** para cifrar y descifrar.

- Ejemplo: **RSA**, Diffie-Hellman.
- **Lenta** (matemáticas pesadas).
- Ventaja: no hace falta compartir un secreto previo.

Por qué importa aquí

Un circuito puede tener **miles de millones** de puertas, y cada una necesita una operación criptográfica. Con criptografía asimétrica sería inviable. Por eso los garbled circuits usan **simétrica (AES)** para las puertas, y reservan la asimétrica para unos pocos pasos del inicio.

El problema concreto que hay que resolver

Ya sabemos que la función es un circuito de puertas. Alicia podría cifrarlo y mandárselo a Bob... pero aparece un obstáculo:

El problema concreto que hay que resolver

Ya sabemos que la función es un circuito de puertas. Alicia podría cifrarlo y mandárselo a Bob... pero aparece un obstáculo:

Si por los cables viajan bits normales

Bob necesita el valor de cada cable para evaluar la siguiente puerta. Pero si ve un 0 o un 1 en el cable de x, **ya conoce la fortuna de Alicia**. Y si ve los cables intermedios, puede deducir todo hacia atrás.

El problema concreto que hay que resolver

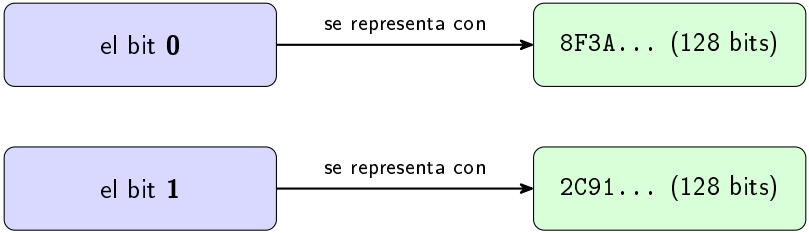
Ya sabemos que la función es un circuito de puertas. Alicia podría cifrarlo y mandárselo a Bob... pero aparece un obstáculo:

Si por los cables viajan bits normales

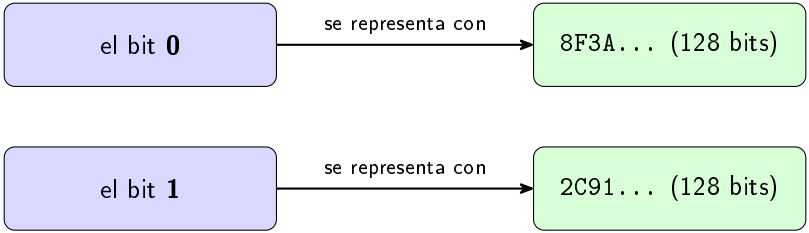
Bob necesita el valor de cada cable para evaluar la siguiente puerta. Pero si ve un 0 o un 1 en el cable de x, **ya conoce la fortuna de Alicia**. Y si ve los cables intermedios, puede deducir todo hacia atrás.

Necesitamos que Bob pueda **evaluar** el circuito
sin nunca saber qué valor lleva cada cable.

La solución: reemplazar cada valor por un disfraz

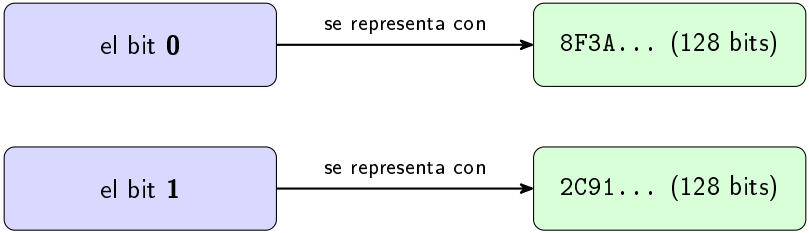


La solución: reemplazar cada valor por un disfraz



Para **cada cable** del circuito, Alicia inventa **dos** cadenas aleatorias: una hará el papel del 0 y la otra el papel del 1. Se llaman **etiquetas** (*labels* o *wire labels*).

La solución: reemplazar cada valor por un disfraz



Para **cada cable** del circuito, Alicia inventa **dos** cadenas aleatorias: una hará el papel del 0 y la otra el papel del 1. Se llaman **etiquetas** (*labels* o *wire labels*).

La propiedad clave

Como son cadenas **completamente aleatorias**, ver una etiqueta no dice absolutamente nada sobre si representa 0 o 1. El significado existe **solo en la tabla privada de Alicia**.

¿De dónde salen las etiquetas exactamente?

Se generan con azar puro

- Alicia pide **128 bits aleatorios** al sistema operativo (`/dev/urandom`, `RAND_bytes()`).
- Debe ser un generador **criptográficamente seguro** (CSPRNG), no el `rand()` común, que es predecible.
- No hay ninguna fórmula ni estructura: es el equivalente digital de lanzar 128 monedas.

¿Por qué DEBEN ser aleatorias?

Si tuvieran cualquier patrón (por ejemplo, que la del 1 siempre empezara con bit 1), Bob deduciría el significado con solo mirarlas y **todo el esquema colapsa**.

¿De dónde salen las etiquetas exactamente?

Se generan con azar puro

- Alicia pide **128 bits aleatorios** al sistema operativo (`/dev/urandom`, `RAND_bytes()`).
- Debe ser un generador **criptográficamente seguro** (CSPRNG), no el `rand()` común, que es predecible.
- No hay ninguna fórmula ni estructura: es el equivalente digital de lanzar 128 monedas.

¿Por qué DEBEN ser aleatorias?

Si tuvieran cualquier patrón (por ejemplo, que la del 1 siempre empezara con bit 1), Bob deduciría el significado con solo mirarlas y **todo el esquema colapsa**.

Etiqueta real (128 bits)	**Abreviada aquí**
`8F3A9C21D4E70B5566A1FE0388C4D912`	`8F3A`

¿Y cómo se evalúa una puerta con etiquetas?

Si por los cables ya no viajan bits sino etiquetas, la tabla de verdad normal no sirve. Hay que **traducir la puerta** al idioma de las etiquetas:

Tabla de verdad normal	$\longrightarrow$	**Tabla cifrada (*garbled table*)**
combinación de bits da un bit de salida		combinación de **etiquetas de entrada** abre una caja con la **etiqueta de salida**

¿Y cómo se evalúa una puerta con etiquetas?

Si por los cables ya no viajan bits sino etiquetas, la tabla de verdad normal no sirve. Hay que **traducir la puerta** al idioma de las etiquetas:

Tabla de verdad normal $\longrightarrow$	**Tabla cifrada (*garbled table*)**
combinación de bits da un bit de salida	combinación de **etiquetas de entrada** abre una caja con la **etiqueta de salida**

La idea de la caja fuerte

Cada fila de la tabla de verdad se convierte en una **caja cerrada con dos candados**: solo se abre si tienes exactamente las dos etiquetas de entrada de esa fila. Dentro está la etiqueta de salida correcta.

Y aquí es donde entra **AES**: es el que cierra y abre esas cajas.

¿Qué es AES?

Advanced Encryption Standard

Cifrador **simétrico** estandarizado en 2001 (concurso público del NIST; diseño belga "Rijndael", de Daemen y Rijmen). Es el estándar mundial vigente: HTTPS, WhatsApp, WiFi, cifrado de disco.

¿Qué es AES?

Advanced Encryption Standard

Cifrador **simétrico** estandarizado en 2001 (concurso público del NIST; diseño belga "Rijndael", de Daemen y Rijmen). Es el estándar mundial vigente: HTTPS, WhatsApp, WiFi, cifrado de disco.

Qué hace

Toma un **bloque de 128 bits** y una **llave**, y los revuelve en 10 rondas hasta producir otros 128 bits que parecen ruido puro. Es

determinista: misma llave + misma entrada $\Rightarrow$ misma salida, siempre.

Efecto avalancha

Cambiar **un solo bit** de la entrada cambia *la mitad* de los bits de salida. `...ccddeeff` $\rightarrow$ `69c4e0d8...`

`...ccddeefe` $\rightarrow$ `3ad77bb4...`

Sin ninguna relación visible entre ambas.

Cómo se usa AES aquí: llave fija

Una sola llave para todo el circuito

Se fija **una llave** k, pública, al inicio del protocolo. No cambia nunca. Las **etiquetas no son la llave**: entran como *dato* que AES revuelve.

$$\text{máscara} \;=\; \text{AES}_k(\text{bloque}) \;\oplus\; \text{bloque}$$

Cómo se usa AES aquí: llave fija

Una sola llave para todo el circuito

Se fija **una llave** k, pública, al inicio del protocolo. No cambia nunca. Las **etiquetas no son la llave**: entran como *dato* que AES revuelve.

$$\text{máscara} = \text{AES}_k(\text{bloque}) \oplus \text{bloque}$$

¿Por qué fija?

Antes de cifrar, AES debe **expandir** la llave en subclaves. Eso es caro. Con llave fija se hace **una sola vez** para todo el circuito, no en cada puerta.

¿Por qué puede ser pública?

Porque AES aquí **no guarda un secreto**: genera ruido determinista. El secreto son las **etiquetas**, que van dentro del bloque de entrada.

Cómo se usa AES aquí: llave fija

Una sola llave para todo el circuito

Se fija **una llave** k, pública, al inicio del protocolo. No cambia nunca. Las **etiquetas no son la llave**: entran como *dato* que AES revuelve.

$$\text{máscara} = \text{AES}_k(\text{bloque}) \oplus \text{bloque}$$

¿Por qué fija?

Antes de cifrar, AES debe **expandir** la llave en subclaves. Eso es caro. Con llave fija se hace **una sola vez** para todo el circuito, no en cada puerta.

¿Por qué puede ser pública?

Porque AES aquí **no guarda un secreto**: genera ruido determinista. El secreto son las **etiquetas**, que van dentro del bloque de entrada.

Demostrado seguro por Bellare, Hoang, Keelveedhi y Rogaway (2013), e implementado en **JustGarble**. Es lo que usan todas las bibliotecas modernas.

Las llaves: quién tiene qué, y por qué

Elemento	**¿Cuántos?**	**¿Quién lo tiene?**	**¿Es secreto?**
Llave k de AES	**1** para todo el circuito	Alicia y Bob	**No, es pública**
Etiquetas por cable	**2** por cada cable	Alicia: las dos Bob: solo una	**Sí**

Las llaves: quién tiene qué, y por qué

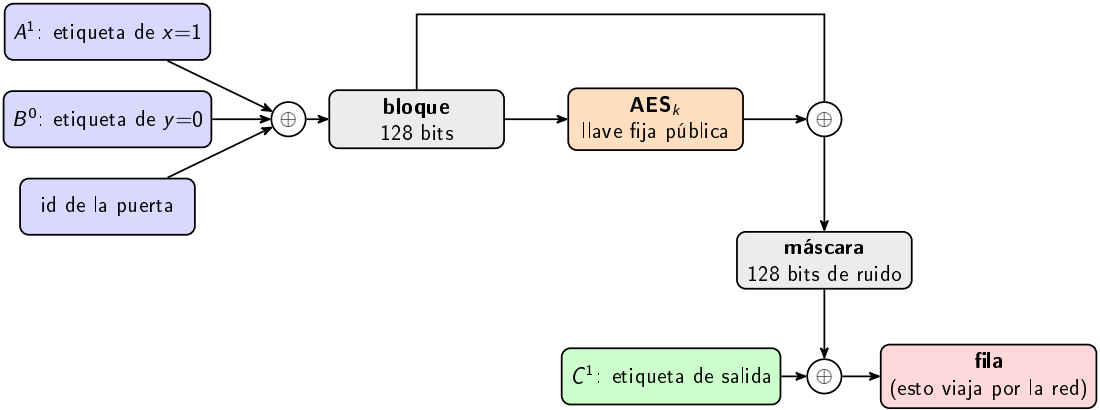
Elemento	¿Cuántos?	¿Quién lo tiene?	¿Es secreto?
Llave k de AES	**1** para todo el circuito	Alicia y Bob	**No, es pública**
Etiquetas por cable	**2** por cada cable	Alicia: las dos Bob: solo una	**Sí**

¿Cómo puede ser pública la llave de AES?

Porque aquí **AES no se usa para guardar un secreto**: se usa como una **máquina de ruido determinista**.

El secreto está en las **etiquetas**, que van dentro del bloque de entrada. Quien no tenga las etiquetas correctas no puede calcular la máscara, aunque conozca k perfectamente.

El pipeline completo, pieza por pieza



El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA

El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale	
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA	
2	**XOR 1/3**: añadir el id	5BDA $\oplus$ 0003	5BD9	**(bloque)**

El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale	
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA	
2	**XOR 1/3**: añadir el id	5BDA $\oplus$ 0003	5BD9	**(bloque)**
3	**AES**$_k$ (única llamada)	5BD9	41B7	

El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale	
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA	
2	**XOR 1/3**: añadir el id	5BDA $\oplus$ 0003	5BD9	**(bloque)**
3	**AES**$_k$ (única llamada)	5BD9	41B7	
4	**XOR 2/3**: Davies-Meyer	41B7 $\oplus$ 5BD9	1A6E	**(máscara)**

El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale	
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA	
2	**XOR 1/3**: añadir el id	5BDA $\oplus$ 0003	5BD9	**(bloque)**
3	**AES**$_k$ (única llamada)	5BD9	41B7	
4	**XOR 2/3**: Davies-Meyer	41B7 $\oplus$ 5BD9	1A6E	**(máscara)**
5	**XOR 3/3**: tapar salida	1A6E $\oplus$ 9AC8	80A6	**(fila)**

El pipeline con valores reales, paso a paso

Ciframos la fila $x{=}1, y{=}0$ de una puerta AND. Etiquetas: $A^1{=}$2C91, $B^0{=}$77D2, salida $C^1{=}$9AC8.

#	Operación	Entra	Sale	
1	**XOR 1/3**: juntar etiquetas	2C91 $\oplus$ 77D2	5BDA	
2	**XOR 1/3**: añadir el id	5BDA $\oplus$ 0003	5BD9	**(bloque)**
3	**AES**$_k$ (única llamada)	5BD9	41B7	
4	**XOR 2/3**: Davies-Meyer	41B7 $\oplus$ 5BD9	1A6E	**(máscara)**
5	**XOR 3/3**: tapar salida	1A6E $\oplus$ 9AC8	80A6	**(fila)**

Bob hace lo mismo, hacia adelante

Con 2C91 y 77D2 repite los pasos 1–4, obtiene la **misma** máscara 1A6E, y calcula 80A6 $\oplus$ 1A6E = 9AC8. **Nunca descifra: regenera y hace XOR.**

¿Por qué tantos XOR? Cada uno tiene su papel

#	**Dónde**	**Para qué sirve**
1	Al armar el bloque	Meter las dos etiquetas y el id de puerta en un solo bloque de 128 bits, para que AES los procese juntos.
2	Después de AES	Volverlo **irreversible**. AES por sí solo se puede deshacer con la llave (que es pública). Con este XOR, despejar el bloque exigiría ya conocer el bloque.
3	Al final	Tapar la etiqueta de salida con la máscara. Es un "candado" perfecto: secreto $\oplus$ ruido $=$ ruido.

¿Por qué tantos XOR? Cada uno tiene su papel

#	**Dónde**	**Para qué sirve**
1	Al armar el bloque	Meter las dos etiquetas y el id de puerta en un solo bloque de 128 bits, para que AES los procese juntos.
2	Después de AES	Volverlo **irreversible**. AES por sí solo se puede deshacer con la llave (que es pública). Con este XOR, despejar el bloque exigiría ya conocer el bloque.
3	Al final	Tapar la etiqueta de salida con la máscara. Es un "candado" perfecto: secreto $\oplus$ ruido $=$ ruido.

XOR se usa tanto porque es **la operación más barata que existe** y es **reversible consigo misma**:

$$(m \oplus k) \oplus k = m.$$

El circuito que vamos a resolver

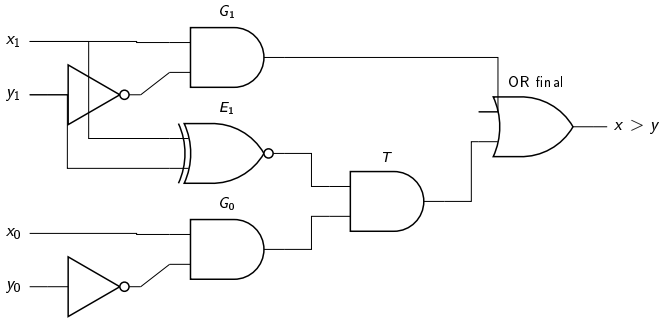
Fortunas de 2 bits: $x = x_1 x_0$, $y = y_1 y_0$ (valores de 0 a 3).

$$x > y \;=\; \underbrace{(x_1 \wedge \neg y_1)}_{G_1} \vee \big(\underbrace{(x_1 \odot y_1)}_{E_1} \wedge \underbrace{(x_0 \wedge \neg y_0)}_{G_0} \big)$$

El circuito que vamos a resolver

Fortunas de 2 bits: $x = x_1 x_0$, $y = y_1 y_0$ (valores de 0 a 3).

$$x > y \;=\; \underbrace{(x_1 \wedge \neg y_1)}_{G_1} \vee \big(\underbrace{(x_1 \odot y_1)}_{E_1} \wedge \underbrace{(x_0 \wedge \neg y_0)}_{G_0}\big)$$



5 puertas a cifrar (G_1, E_1, G_0, T, OR). Los dos NOT son **gratis**.

Los valores secretos

Alicia: $x = 10_2 = 2$ Bob: $y = 01_2 = 1$

Los valores secretos

Alicia: $x = 10_2 = 2$ Bob: $y = 01_2 = 1$

Bits de Alicia	Bits de Bob
$x_1 = 1$	$y_1 = 0$
$x_0 = 0$	$y_0 = 1$

Los valores secretos

Alicia: $x = 10_2 = 2$ Bob: $y = 01_2 = 1$

Bits de Alicia	Bits de Bob
$x_1 = 1$	$y_1 = 0$
$x_0 = 0$	$y_0 = 1$

Resultado esperado: $2 > 1$, o sea **verdadero**.
Pero ellos no lo saben: lo vamos a descubrir puerta por puerta, a ciegas.

Paso 1: Alicia genera TODAS las etiquetas

Dos cadenas aleatorias por cada cable del circuito (9 cables):

Cable	**Qué lleva**	**Etiqueta del 0**	**Etiqueta del 1**
x_1	bit alto de Alicia	8F3A	2C91
y_1	bit alto de Bob	77D2	E04B
x_0	bit bajo de Alicia	4D6A	B183
y_0	bit bajo de Bob	9E7C	03F5
G_1	¿gano en el bit alto?	5512	9AC8
E_1	¿empate en el bit alto?	1AF0	6C2E
G_0	¿gano en el bit bajo?	55A1	C0DE
T	empate alto Y gano bajo	7734	A21F
salida	$x > y$	0F3C	8AD5

Paso 1: Alicia genera TODAS las etiquetas

Dos cadenas aleatorias por cada cable del circuito (9 cables):

Cable	Qué lleva	Etiqueta del 0	Etiqueta del 1
x_1	bit alto de Alicia	8F3A	2C91
y_1	bit alto de Bob	77D2	E04B
x_0	bit bajo de Alicia	4D6A	B183
y_0	bit bajo de Bob	9E7C	03F5
G_1	¿gano en el bit alto?	5512	9AC8
E_1	¿empate en el bit alto?	1AF0	6C2E
G_0	¿gano en el bit bajo?	55A1	C0DE
T	empate alto Y gano bajo	7734	A21F
salida	$x > y$	0F3C	8AD5

Esta tabla completa nunca sale de la computadora de Alicia.

Todas las puertas del circuito, con sus etiquetas

#	Puerta	Entrada A (0 / 1)	Entrada B (0 / 1)	Salida (0 / 1)
1	$G_1 = x_1 \wedge \neg y_1$	8F3A / 2C91	77D2 / E04B	5512 / 9AC8
2	$E_1 = x_1 \odot y_1$	8F3A / 2C91	77D2 / E04B	1AF0 / 6C2E
3	$G_0 = x_0 \wedge \neg y_0$	4D6A / B183	9E7C / 03F5	55A1 / C0DE
4	$T = E_1 \wedge G_0$	1AF0 / 6C2E	55A1 / C0DE	7734 / A21F
5	$x{>}y = G_1 \vee T$	5512 / 9AC8	7734 / A21F	0F3C / 8AD5

Todas las puertas del circuito, con sus etiquetas

#	Puerta	Entrada A (0 / 1)	Entrada B (0 / 1)	Salida (0 / 1)
1	$G_1 = x_1 \wedge \neg y_1$	8F3A / 2C91	77D2 / E04B	5512 / 9AC8
2	$E_1 = x_1 \odot y_1$	8F3A / 2C91	77D2 / E04B	1AF0 / 6C2E
3	$G_0 = x_0 \wedge \neg y_0$	4D6A / B183	9E7C / 03F5	55A1 / C0DE
4	$T = E_1 \wedge G_0$	1AF0 / 6C2E	55A1 / C0DE	7734 / A21F
5	$x{>}y = G_1 \vee T$	5512 / 9AC8	7734 / A21F	0F3C / 8AD5

Lo importante de esta tabla

Fíjense en las puertas 4 y 5: sus **entradas** son las **salidas** de las puertas anteriores. Las mismas cadenas reaparecen. **Así se encadena el circuito.**

Paso 2: cifrar la puerta G_1 — tabla de verdad

$G_1 = x_1 \wedge \neg y_1$. Primero la lógica, con el NOT ya absorbido:

x_1	y_1	$x_1 \wedge \neg y_1$	**Etiqueta de salida**
0	0	0	`5512`
0	1	0	`5512`
1	0	**1**	`9AC8`
1	1	0	`5512`

Paso 2: cifrar la puerta G_1 — tabla de verdad

$G_1 = x_1 \wedge \neg y_1$. Primero la lógica, con el NOT ya absorbido:

x_1	y_1	$x_1 \wedge \neg y_1$	**Etiqueta de salida**
0	0	0	`5512`
0	1	0	`5512`
1	0	**1**	`9AC8`
1	1	0	`5512`

El NOT es gratis

Alicia no crea ninguna puerta NOT: simplemente construye la tabla ya con y_1 invertido. Bob nunca se entera de que hubo un NOT.

Paso 3: cada fila se convierte en una caja cerrada

Caso	Llaves (etiquetas de entrada)	Dentro	AES	Fila cifrada
$x_1{=}0, y_1{=}0$	8F3A + 77D2	5512	$\rightarrow$	03D8
$x_1{=}0, y_1{=}1$	8F3A + E04B	5512	$\rightarrow$	6C12
$x_1{=}1, y_1{=}0$	2C91 + 77D2	9AC8	$\rightarrow$	BE59
$x_1{=}1, y_1{=}1$	2C91 + E04B	5512	$\rightarrow$	F7A2

Paso 3: cada fila se convierte en una caja cerrada

Caso	Llaves (etiquetas de entrada)	Dentro	AES	Fila cifrada
$x_1{=}0, y_1{=}0$	8F3A + 77D2	5512	$\rightarrow$	03D8
$x_1{=}0, y_1{=}1$	8F3A + E04B	5512	$\rightarrow$	6C12
$x_1{=}1, y_1{=}0$	2C91 + 77D2	9AC8	$\rightarrow$	BE59
$x_1{=}1, y_1{=}1$	2C91 + E04B	5512	$\rightarrow$	F7A2



Paso 4: barajar las filas

Antes (en orden)

03D8 ← caso 0, 0
6C12 ← caso 0, 1
BE59 ← caso 1, 0
F7A2 ← caso 1, 1

La **posición** delataría los bits.

Paso 4: barajar las filas

Antes (en orden)

`03D8` ← caso $0,0$
`6C12` ← caso $0,1$
`BE59` ← caso $1,0$
`F7A2` ← caso $1,1$

La **posición** delataría los bits.

Después (barajadas)

`F7A2`
`03D8`
`BE59`
`6C12`

Cuatro cadenas sin orden ni rótulos. **Esto es la puerta cifrada.**

Paso 4: barajar las filas

Antes (en orden)

`03D8` ← caso $0,0$
`6C12` ← caso $0,1$
`BE59` ← caso $1,0$
`F7A2` ← caso $1,1$

La **posición** delataría los bits.

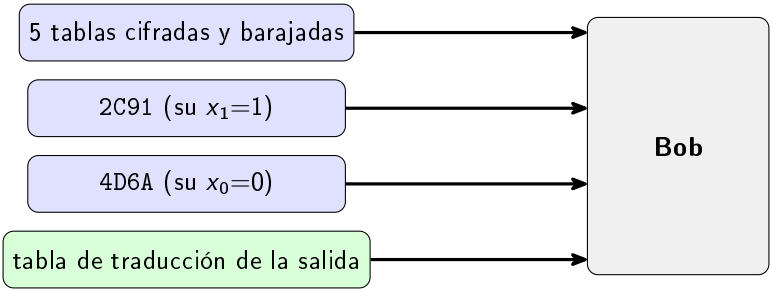
Después (barajadas)

`F7A2`
`03D8`
`BE59`
`6C12`

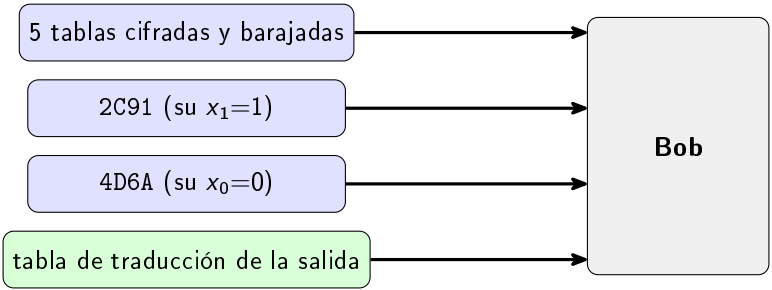
Cuatro cadenas sin orden ni rótulos. **Esto es la puerta cifrada.**

Alicia repite los pasos 2–4 para las **5 puertas**. Todo este trabajo es **local**: aún no ha enviado nada.

Paso 5: Alicia envía el paquete



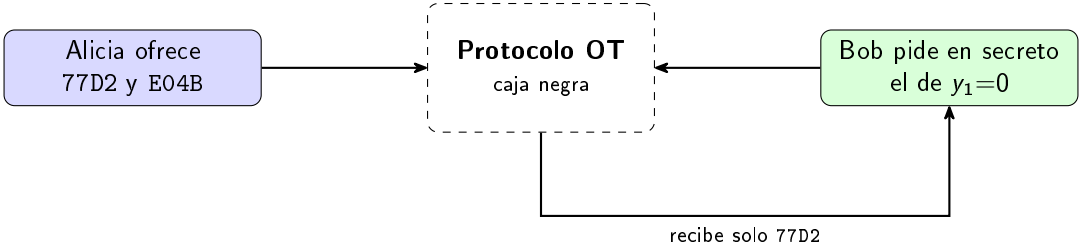
Paso 5: Alicia envía el paquete



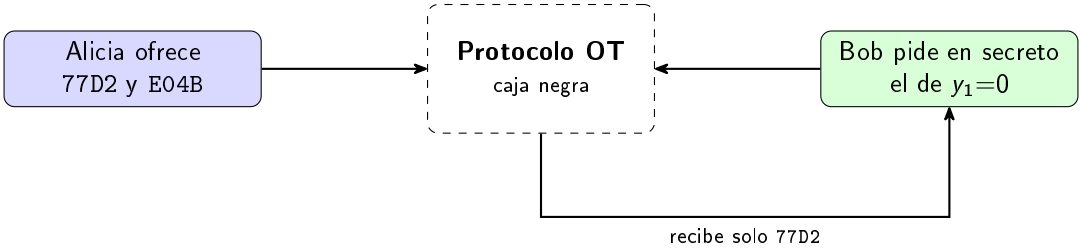
Lo que Bob percibe

Un montón de cadenas hexadecimales indistinguibles del ruido. No sabe si 2C91 significa 0 o 1, ni qué fila es cuál, ni qué valores circulan.

Paso 6: Bob consigue SUS etiquetas (Oblivious Transfer)



Paso 6: Bob consigue SUS etiquetas (Oblivious Transfer)



Lo que garantiza

Bob recibe **exactamente** la etiqueta de su bit. Alicia **no sabe cuál pidió**. Bob **no aprende nada** de la otra.

Detalles

Un OT por **cada bit** de Bob (aquí 2: y_1 y y_0). Se construye con criptografía **asimétrica** (RSA o curvas elípticas): es la parte lenta, por eso se usa poco.

Paso 7: Bob evalúa la puerta G_1

Bob tiene `2C91` (de Alicia) y `77D2` (del OT). Prueba las 4 cajas:

Caja	Regenera máscara y hace XOR	Sale	
`F7A2`	$\rightarrow$	`k#T9%wq2...`	basura
`03D8`	$\rightarrow$	`j@4Lp!x8...`	basura
`BE59`	$\rightarrow$	**OK** $\|\|$ `9AC8`	¡abrió!
`6C12`	$\rightarrow$	`zR7&mN0c...`	basura

Paso 7: Bob evalúa la puerta G_1

Bob tiene 2C91 (de Alicia) y 77D2 (del OT). Prueba las 4 cajas:

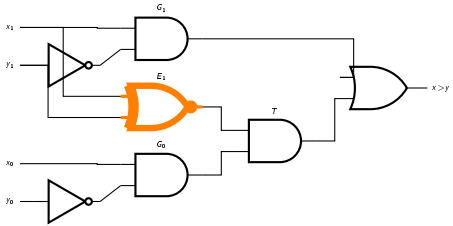
Caja	Regenera máscara y hace XOR	Sale	
F7A2	$\rightarrow$	k#T9%wq2...	basura
03D8	$\rightarrow$	j@4Lp!x8...	basura
BE59	$\rightarrow$	**OK** $\|\|$ 9AC8	¡abrió!
6C12	$\rightarrow$	zR7&mN0c...	basura

Lo que Bob sabe ahora

Que tiene la cadena 9AC8. **Nada más.** No sabe qué fila abrió (estaban barajadas), ni si 9AC8 representa 0 o 1.

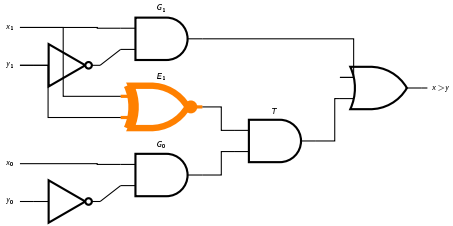
(En la práctica, la técnica *point-and-permute* le dice directamente cuál caja abrir, sin probar las cuatro.)

Paso 8a: puerta 2 — E_1 (¿empatamos arriba?)



Entra: `2C91` (x_1) y `77D2` (y_1) — *las mismas que la puerta 1, otro cable de salida*

Paso 8a: puerta 2 — E_1 (¿empatamos arriba?)

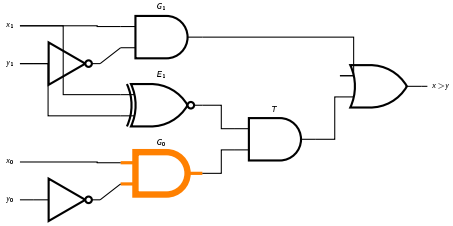


Entra: `2C91` (x_1) y `77D2` (y_1) — *las mismas que la puerta 1, otro cable de salida*

Paso	Operación	Resultado
XOR 1	`2C91` $\oplus$ `77D2` $\oplus$ `0002`	`5BD8` (bloque)
AES	AES_k(`5BD8`)	`C104`
XOR 2	`C104` $\oplus$ `5BD8`	`9EDC` (máscara)
XOR 3	`802C` $\oplus$ `9EDC`	`1AF0`

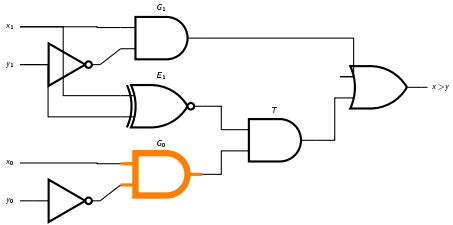
Sale `1AF0`. *Alicia sabe que es E_1=0 (no hay empate); Bob no.*

Paso 8b: puerta 3 — G_0 (¿gano abajo?)



Entra: `4D6A` (x_0=0, de Alicia) y `03F5` (y_0=1, **segundo OT**)

Paso 8b: puerta 3 — G_0 (¿gano abajo?)

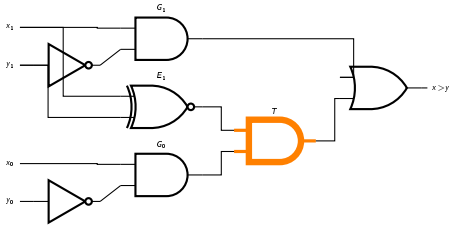


Entra: `4D6A` (x_0=0, de Alicia) y `03F5` (y_0=1, **segundo OT**)

Paso	**Operación**	**Resultado**
XOR 1	`4D6A` $\oplus$ `03F5` $\oplus$ `0003`	`4E9C` (bloque)
AES	AES_k(`4E9C`)	`B2D1`
XOR 2	`B2D1` $\oplus$ `4E9C`	`FC4D` (máscara)
XOR 3	`A9EC` $\oplus$ `FC4D`	`55A1`

Sale `55A1`, que es G_0=0.

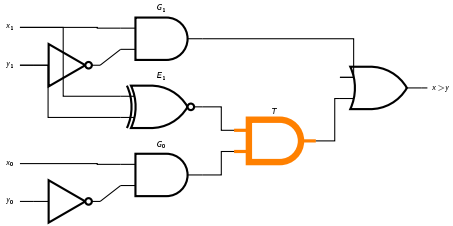
Paso 8c: puerta 4 — T (aquí se encadena)



Las entradas ya no vienen de nadie

Entra `1AF0` (salida de la puerta 2) y `55A1` (salida de la puerta 3). **Nadie se las mandó y no hubo OT: las produjo Bob evaluando.**

Paso 8c: puerta 4 — T (aquí se encadena)

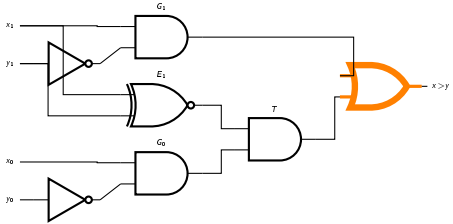


Las entradas ya no vienen de nadie

Entra `1AF0` (salida de la puerta 2) y `55A1` (salida de la puerta 3). **Nadie se las mandó y no hubo OT: las produjo Bob evaluando.**

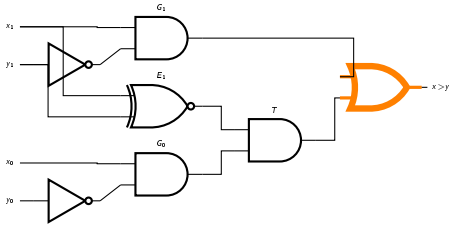
Paso	Operación	Resultado
XOR 1	`1AF0` $\oplus$ `55A1` $\oplus$ `0004`	`4F55` (bloque)
AES	AES_k(`4F55`)	`7A93`
XOR 2	`7A93` $\oplus$ `4F55`	`35C6` (máscara)
XOR 3	`42F2` $\oplus$ `35C6`	`7734`

Paso 8d: puerta 5 — el OR final



Entra: `9AC8` (salida puerta 1) y `7734` (salida puerta 4)

Paso 8d: puerta 5 — el OR final



Entra: `9AC8` (salida puerta 1) y `7734` (salida puerta 4)

Paso	Operación	Resultado
XOR 1	`9AC8` $\oplus$ `7734` $\oplus$ `0005`	`EDF9` (bloque)
AES	AES_k(`EDF9`)	`2C40`
XOR 2	`2C40` $\oplus$ `EDF9`	`C1B9` (máscara)
XOR 3	`4B6C` $\oplus$ `C1B9`	`8AD5`

Sale `8AD5`: la última etiqueta del circuito. Bob evaluó 5 puertas y sigue teniendo solo una cadena hexadecimal.

Paso 9: la traducción final

Bob terminó con `8AD5`. Consulta la tabla de traducción que Alicia le envió al principio:

$$\texttt{0F3C} \longrightarrow \mathbf{0} \quad \text{(no es mayor)}$$

$$\texttt{8AD5} \longrightarrow \mathbf{1} \quad \text{(sí es mayor)}$$

Paso 9: la traducción final

Bob terminó con `8AD5`. Consulta la tabla de traducción que Alicia le envió al principio:

$$\texttt{0F3C} \longrightarrow \mathbf{0} \quad \text{(no es mayor)}$$

$$\texttt{8AD5} \longrightarrow \mathbf{1} \quad \text{(sí es mayor)}$$

$$x > y = 1$$

Paso 9: la traducción final

Bob terminó con `8AD5`. Consulta la tabla de traducción que Alicia le envió al principio:

$$\texttt{0F3C} \longrightarrow \mathbf{0} \quad \text{(no es mayor)}$$

$$\texttt{8AD5} \longrightarrow \mathbf{1} \quad \text{(sí es mayor)}$$

$$x > y = 1$$

Ese es el **único** bit que salió de las 5 puertas cifradas. Todo lo demás quedó oculto para siempre.

Auditoría: ¿quién terminó sabiendo qué?

Información	**¿Alicia?**	**¿Bob?**
Los bits de x	Sí (son suyos)	No
Los bits de y	**No**	Sí (son suyos)
El resultado $x > y$	Sí	Sí
Valores intermedios (G_1, E_1, G_0, T)	Los cifró, no vio la evaluación	Los evaluó cifrados
Qué fila abrió Bob en cada puerta	**No**	Sí, pero no sabe qué sign

Auditoría: ¿quién terminó sabiendo qué?

Información	¿Alicia?	¿Bob?
Los bits de x	Sí (son suyos)	No
Los bits de y	**No**	Sí (son suyos)
El resultado $x > y$	Sí	Sí
Valores intermedios (G_1, E_1, G_0, T)	Los cifró, no vio la evaluación	Los evaluó cifrados
Qué fila abrió Bob en cada puerta	**No**	Sí, pero no sabe qué sign

La prueba del algodón

Si x hubiera sido otro valor, lo **único** distinto para Bob es que habría recibido otras cadenas aleatorias. Su experiencia es **idéntica** en todos los universos posibles. Eso *es* la demostración de seguridad.

¿Por qué circuitos lógicos? El argumento decisivo

Imagina cifrar la función **entera** como una sola tabla de verdad gigante, en vez de descomponerla en puertas:

Comparador	**Tabla única (filas)**	**Con puertas (filas)**
2 bits	$2^4 = 16$	$5 \times 4 = 20$

¿Por qué circuitos lógicos? El argumento decisivo

Imagina cifrar la función **entera** como una sola tabla de verdad gigante, en vez de descomponerla en puertas:

Comparador	**Tabla única (filas)**	**Con puertas (filas)**
2 bits	$2^4 = 16$	$5 \times 4 = 20$
4 bits	$2^8 = 256$	$16 \times 4 = 64$
8 bits	$2^{16} = 65{,}536$	$35 \times 4 = 140$

¿Por qué circuitos lógicos? El argumento decisivo

Imagina cifrar la función **entera** como una sola tabla de verdad gigante, en vez de descomponerla en puertas:

Comparador	**Tabla única (filas)**	**Con puertas (filas)**
2 bits	$2^4 = 16$	$5 \times 4 = 20$
4 bits	$2^8 = 256$	$16 \times 4 = 64$
8 bits	$2^{16} = 65{,}536$	$35 \times 4 = 140$
16 bits	$2^{32} \approx 4$ mil millones	$\sim 70 \times 4 = 280$

¿Por qué circuitos lógicos? El argumento decisivo

Imagina cifrar la función **entera** como una sola tabla de verdad gigante, en vez de descomponerla en puertas:

Comparador	**Tabla única (filas)**	**Con puertas (filas)**
2 bits	$2^4 = 16$	$5 \times 4 = 20$
4 bits	$2^8 = 256$	$16 \times 4 = 64$
8 bits	$2^{16} = 65{,}536$	$35 \times 4 = 140$
16 bits	$2^{32} \approx 4$ mil millones	$\sim 70 \times 4 = 280$
64 bits	$2^{128} \approx 3.4 \times 10^{38}$	$\sim 300 \times 4 = 1{,}200$

¿Por qué circuitos lógicos? El argumento decisivo

Imagina cifrar la función **entera** como una sola tabla de verdad gigante, en vez de descomponerla en puertas:

Comparador	**Tabla única (filas)**	**Con puertas (filas)**
2 bits	$2^4 = 16$	$5 \times 4 = 20$
4 bits	$2^8 = 256$	$16 \times 4 = 64$
8 bits	$2^{16} = 65{,}536$	$35 \times 4 = 140$
16 bits	$2^{32} \approx 4$ mil millones	$\sim 70 \times 4 = 280$
64 bits	$2^{128} \approx 3.4 \times 10^{38}$	$\sim 300 \times 4 = 1{,}200$

Exponencial contra lineal

La tabla única crece **exponencialmente**: con 64 bits necesitaría más filas que átomos hay en la Tierra. **Es físicamente imposible.**

Descompuesto en puertas crece **linealmente**: 1,200 filas. **Cabe en 38 kilobytes.**

La razón de fondo

Una puerta lógica es **la operación más pequeña que existe**.

La razón de fondo

Una puerta lógica es **la operación más pequeña que existe**.

Dos entradas, siempre

$2^2 = 4$ combinaciones posibles.

Por eso la tabla cifrada de **cualquier** puerta es **siempre de 4 filas**, sin importar qué tan complejo sea el problema global.

La razón de fondo

Una puerta lógica es **la operación más pequeña que existe**.

Dos entradas, siempre

$2^2 = 4$ combinaciones posibles.

Por eso la tabla cifrada de **cualquier** puerta es **siempre de 4 filas**, sin importar qué tan complejo sea el problema global.

Y encaja cualquier función

Desde los años 30 se sabe que **toda** función computable se puede escribir como circuito booleano.

Yao no invento eso: lo tomo prestado. Le dio un lenguaje universal donde cabe cualquier problema.

La razón de fondo

Una puerta lógica es **la operación más pequeña que existe**.

Dos entradas, siempre

$2^2 = 4$ combinaciones posibles.

Por eso la tabla cifrada de **cualquier** puerta es **siempre de 4 filas**, sin importar qué tan complejo sea el problema global.

Y encaja cualquier función

Desde los años 30 se sabe que **toda** función computable se puede escribir como circuito booleano.

Yao no invento eso: lo tomo prestado. Le dio un lenguaje universal donde cabe cualquier problema.

Esa granularidad fina es lo que vuelve viable toda la técnica.

¿Qué pasa con más bits?

Lo que NUNCA crece

La tabla de una puerta es **siempre de 4 filas**.

Una puerta de 2 entradas tiene $2^2 = 4$ combinaciones. Punto, sin importar el tamaño del problema.

Lo que SÍ crece

El **número de puertas** y los **OT**.

Comparador	**Puertas**	**OT**
2 bits	5	2
4 bits	16	4
8 bits	∼35	8
64 bits	∼300	64

¿Qué pasa con más bits?

Lo que NUNCA crece

La tabla de una puerta es **siempre de 4 filas**.

Una puerta de 2 entradas tiene $2^2 = 4$ combinaciones. Punto, sin importar el tamaño del problema.

Lo que SÍ crece

El **número de puertas** y los **OT**.

Comparador	**Puertas**	**OT**
2 bits	5	2
4 bits	16	4
8 bits	∼35	8
64 bits	∼300	64

Crece de forma **lineal**, no exponencial. Por eso la técnica escala a circuitos de miles de millones de puertas.

Cuántos datos viajan por la red

Cada puerta AND son **2 filas** de 128 bits (16 bytes) = **32 bytes**. Las XOR son **gratis**: 0 bytes.

Cálculo	**Puertas AND**	**Datos**	**En red de 100 Mbps**
Comparar 2 bits (nuestro ejemplo)	5	160 B	instantáneo
Comparar 64 bits	∼300	9.6 KB	instantáneo

Cuántos datos viajan por la red

Cada puerta AND son **2 filas** de 128 bits (16 bytes) = **32 bytes**. Las XOR son **gratis**: 0 bytes.

Cálculo	**Puertas AND**	**Datos**	**En red de 100 Mbps**
Comparar 2 bits (nuestro ejemplo)	5	160 B	instantáneo
Comparar 64 bits	∼300	9.6 KB	instantáneo
Sumar dos números de 64 bits	63	2 KB	instantáneo
Multiplicar dos de 64 bits	∼4,000	128 KB	0.01 s

Cuántos datos viajan por la red

Cada puerta AND son **2 filas** de 128 bits (16 bytes) = **32 bytes**. Las XOR son **gratis**: 0 bytes.

Cálculo	**Puertas AND**	**Datos**	**En red de 100 Mbps**
Comparar 2 bits (nuestro ejemplo)	5	160 B	instantáneo
Comparar 64 bits	∼300	9.6 KB	instantáneo
Sumar dos números de 64 bits	63	2 KB	instantáneo
Multiplicar dos de 64 bits	∼4,000	128 KB	0.01 s
Un cifrado AES completo	∼6,400	205 KB	0.02 s
Comparar dos genomas (1M posiciones)	∼300 M	**9.6 GB**	13 min

Cuántos datos viajan por la red

Cada puerta AND son **2 filas** de 128 bits (16 bytes) = **32 bytes**. Las XOR son **gratis**: 0 bytes.

Cálculo	**Puertas AND**	**Datos**	**En red de 100 Mbps**
Comparar 2 bits (nuestro ejemplo)	5	160 B	instantáneo
Comparar 64 bits	∼300	9.6 KB	instantáneo
Sumar dos números de 64 bits	63	2 KB	instantáneo
Multiplicar dos de 64 bits	∼4,000	128 KB	0.01 s
Un cifrado AES completo	∼6,400	205 KB	0.02 s
Comparar dos genomas (1M posiciones)	∼300 M	**9.6 GB**	13 min
Inferencia en una red neuronal	miles de millones	**decenas de GB**	horas

Cuántos datos viajan por la red

Cada puerta AND son **2 filas** de 128 bits (16 bytes) = **32 bytes**. Las XOR son **gratis**: 0 bytes.

Cálculo	**Puertas AND**	**Datos**	**En red de 100 Mbps**
Comparar 2 bits (nuestro ejemplo)	5	160 B	instantáneo
Comparar 64 bits	∼300	9.6 KB	instantáneo
Sumar dos números de 64 bits	63	2 KB	instantáneo
Multiplicar dos de 64 bits	∼4,000	128 KB	0.01 s
Un cifrado AES completo	∼6,400	205 KB	0.02 s
Comparar dos genomas (1M posiciones)	∼300 M	**9.6 GB**	13 min
Inferencia en una red neuronal	miles de millones	**decenas de GB**	horas

El cuello de botella real no es el CPU: es la red

Cifrar es rapidísimo (nanosegundos por puerta). Lo que duele es **mover los bytes**. Por eso todas las optimizaciones históricas atacaron el tamaño de la tabla: de 4 filas a 3, a 2, a 1.5.

Los otros dos problemas de escalar

Memoria

Guardar el circuito entero es inviable (decenas de GB).

Solución: **pipelining**. Alicia cifra y envía al vuelo, Bob evalúa y descarta. El circuito completo **nunca existe** en memoria.

Un solo uso

Reutilizar etiquetas rompe la seguridad: Bob acumularía las dos llaves del mismo cable y podría abrir cajas de más.

Sin solución barata: hay que cifrar todo de nuevo en cada ejecución.

Los otros dos problemas de escalar

Memoria

Guardar el circuito entero es inviable (decenas de GB).

Solución: **pipelining**. Alicia cifra y envía al vuelo, Bob evalúa y descarta. El circuito completo **nunca existe** en memoria.

Un solo uso

Reutilizar etiquetas rompe la seguridad: Bob acumularía las dos llaves del mismo cable y podría abrir cajas de más.

Sin solución barata: hay que cifrar todo de nuevo en cada ejecución.

Las optimizaciones históricas (point-and-permute 1990, free-XOR 2008, half-gates 2015, three-halves 2021) redujeron el tamaño de cada puerta AND **de 4 textos cifrados a 1.5**.

El modelo semi-honesto (y su límite)

Lo que hemos supuesto hasta ahora

Ambos **siguen las reglas** del protocolo, pero intentan deducir lo que puedan de lo que ven. Se llama modelo **semi-honesto** (*honest-but-curious*).

El modelo semi-honesto (y su límite)

Lo que hemos supuesto hasta ahora

Ambos **siguen las reglas** del protocolo, pero intentan deducir lo que puedan de lo que ven. Se llama modelo **semi-honesto** (*honest-but-curious*).

El problema: ¿y si mienten?

El protocolo básico de Yao **no protege** contra alguien que se desvíe del protocolo a propósito. Ahí se llama modelo **malicioso** (o activo), y hacen falta defensas extra.

El modelo semi-honesto (y su límite)

Lo que hemos supuesto hasta ahora

Ambos **siguen las reglas** del protocolo, pero intentan deducir lo que puedan de lo que ven. Se llama modelo **semi-honesto** (*honest-but-curious*).

El problema: ¿y si mienten?

El protocolo básico de Yao **no protege** contra alguien que se desvíe del protocolo a propósito. Ahí se llama modelo **malicioso** (o activo), y hacen falta defensas extra.

Veamos exactamente **cómo** puede hacer trampa cada uno.

Cómo puede hacer trampa cada parte

Alicia (garbler) — la peligrosa

Ella construye el circuito, así que puede cifrar **uno distinto al acordado**:

- En vez de $x > y$, cifrar un circuito que **revele directamente** el primer bit de y.
- Bob lo evaluaría sin sospechar nada: para él todo es ruido.

Esta es la trampa grave del protocolo.

Cómo puede hacer trampa cada parte

Alicia (garbler) — la peligrosa

Ella construye el circuito, así que puede cifrar **uno distinto al acordado**:

- En vez de $x > y$, cifrar un circuito que **revele directamente** el primer bit de y.
- Bob lo evaluaría sin sospechar nada: para él todo es ruido.

Esta es la trampa grave del protocolo.

Bob (evaluator) — más limitado

- Puede **mentir sobre el resultado** que le reporta a Alicia (decir 0 cuando fue 1).
- Puede intentar romper el OT para conseguir *las dos* etiquetas de un cable, pero eso equivale a romper RSA.
- Puede **abandonar** el protocolo a medias tras ver el resultado.

Cómo puede hacer trampa cada parte

Alicia (garbler) — la peligrosa

Ella construye el circuito, así que puede cifrar **uno distinto al acordado**:

- En vez de $x > y$, cifrar un circuito que **revele directamente** el primer bit de y.
- Bob lo evaluaría sin sospechar nada: para él todo es ruido.

Esta es la trampa grave del protocolo.

Bob (evaluator) — más limitado

- Puede **mentir sobre el resultado** que le reporta a Alicia (decir 0 cuando fue 1).
- Puede intentar romper el OT para conseguir *las dos* etiquetas de un cable, pero eso equivale a romper RSA.
- Puede **abandonar** el protocolo a medias tras ver el resultado.

La asimetría es clara: **quien cifra tiene más poder para hacer trampa** que quien evalúa.

¿Cómo detecta Bob un circuito malicioso?

El problema

Evaluando **no puede**: todo es ruido para él. Necesita **abrir** el circuito y revisarlo... pero si Alicia le da las etiquetas para abrirlo, Bob puede evaluar con *ambos* valores de cada cable y aprender la entrada de Alicia.

¿Cómo detecta Bob un circuito malicioso?

El problema

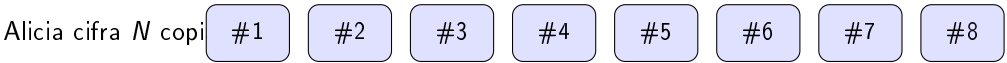
Evaluando **no puede**: todo es ruido para él. Necesita **abrir** el circuito y revisarlo... pero si Alicia le da las etiquetas para abrirlo, Bob puede evaluar con *ambos* valores de cada cable y aprender la entrada de Alicia.

La solución: un circuito **o** se abre **o** se evalúa, nunca ambos

Alicia cifra **muchas copias**. Bob elige al azar cuáles abrir.

Para las copias que se abren, Alicia revela **la semilla aleatoria** con la que las generó. Con esa semilla Bob **regenera todo**: las etiquetas, las tablas, el barajado. Si lo regenerado no coincide bit por bit con lo que recibió, o si el circuito no corresponde a la función pactada, **Alicia queda descubierta**.

Cut-and-choose



Las que abre

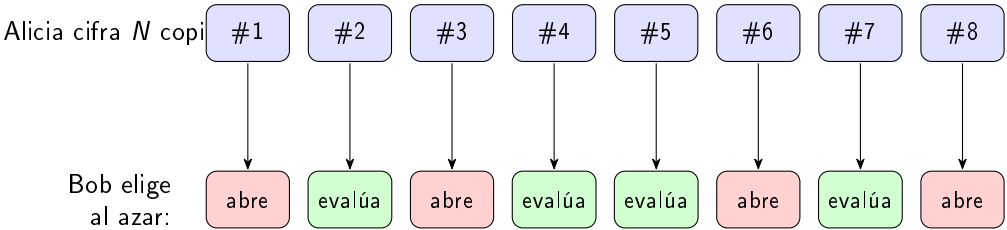
Verifica con la semilla. Si alguna está mal, **aborta**.

Las que evalúa

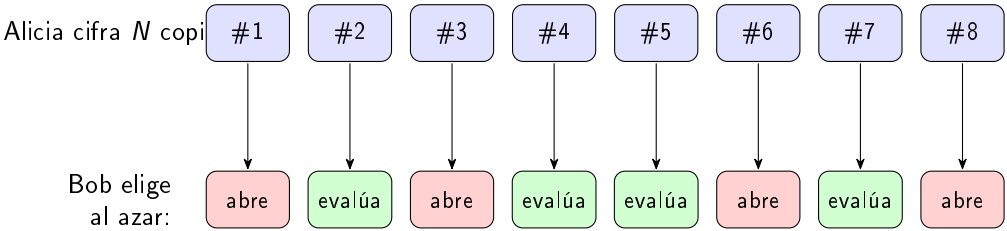
Toma el resultado **mayoritario** (si Alicia coló una mala, queda en minoría).

Alicia **no sabe de antemano** cuáles abrirá Bob: hacer trampa se vuelve una apuesta perdedora.

Cut-and-choose



Cut-and-choose



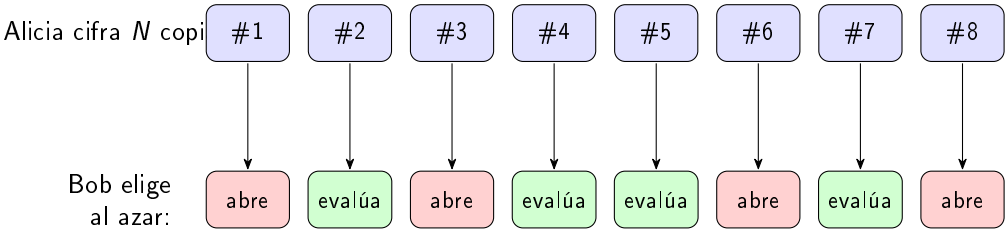
Las que abre

Verifica con la semilla. Si alguna está mal, **aborta**.

Las que evalúa

Toma el resultado **mayoritario** (si Alicia coló una mala, queda en minoría).

Cut-and-choose



Las que abre

Verifica con la semilla. Si alguna está mal, **aborta**.

Las que evalúa

Toma el resultado **mayoritario** (si Alicia coló una mala, queda en minoría).

Alicia **no sabe de antemano** cuáles abrirá Bob: hacer trampa se vuelve una apuesta perdedora.

Más bits no dan más seguridad criptográfica

Cuidado con esta confusión

La **seguridad del protocolo** no depende del número de bits de las fortunas: depende del tamaño de las **etiquetas** (128 bits) y del modelo de adversario.

Más bits no dan más seguridad criptográfica

Cuidado con esta confusión

La **seguridad del protocolo** no depende del número de bits de las fortunas: depende del tamaño de las **etiquetas** (128 bits) y del modelo de adversario.

Lo que sí mejora con más bits: la privacidad del resultado

Tamaño	**Si $y = 1$ y "$x > y$"**	**Bob deduce**
2 bits	$x \in \{2, 3\}$	2 de 4 valores
4 bits	$x \in \{2, ..., 15\}$	14 de 16 valores
64 bits	$x \in \{2, ..., 2^{64}\}$	prácticamente nada

Más bits no dan más seguridad criptográfica

Cuidado con esta confusión

La **seguridad del protocolo** no depende del número de bits de las fortunas: depende del tamaño de las **etiquetas** (128 bits) y del modelo de adversario.

Lo que sí mejora con más bits: la privacidad del resultado

Tamaño	**Si $y = 1$ y "$x > y$"**	**Bob deduce**
2 bits	$x \in \{2, 3\}$	2 de 4 valores
4 bits	$x \in \{2, ..., 15\}$	14 de 16 valores
64 bits	$x \in \{2, ..., 2^{64}\}$	prácticamente nada

Con pocos bits, **el resultado mismo** filtra mucho. Eso no es culpa del protocolo: pasaría igual con un notario. Con muchos bits, esa fuga se vuelve insignificante.

Caso real: Meta — *Private Lift Measurement*

El problema de negocio

Una marca pone anuncios en Instagram y quiere saber **cuántas ventas le generaron**. Para eso hay que cruzar dos bases:

- **Meta** tiene quién vio el anuncio.
- **El anunciante** tiene quién compró.

Cruzarlas normalmente significa que uno le entrega su lista de usuarios al otro. **Ninguno quiere hacerlo**, y la ley cada vez lo permite menos.

Caso real: Meta — *Private Lift Measurement*

El problema de negocio

Una marca pone anuncios en Instagram y quiere saber **cuántas ventas le generaron**. Para eso hay que cruzar dos bases:

- **Meta** tiene quién vio el anuncio.
- **El anunciante** tiene quién compró.

Cruzarlas normalmente significa que uno le entrega su lista de usuarios al otro. **Ninguno quiere hacerlo**, y la ley cada vez lo permite menos.

La solución

Es **literalmente** el problema de los millonarios: dos empresas, datos privados, un solo número como resultado. Meta lo resolvió con **computación de dos partes**.

Los datos concretos del caso

Empresa	Meta (Facebook)
Producto	Private Lift Measurement (PLM)
Inicio	2020, en pruebas con socios seleccionados
Estado	2023: usado por algunos de sus mayores anunciantes

Los datos concretos del caso

Empresa	Meta (Facebook)
Producto	Private Lift Measurement (PLM)
Inicio	2020, en pruebas con socios seleccionados
Estado	2023: usado por algunos de sus mayores anunciantes
Código	Abierto: `github.com/facebookresearch/fbpcf`
Infraestructura	AWS, contenedores que se levantan por ejecución
Simetría	El anunciante despliega **el mismo** software que Meta

Los datos concretos del caso

Empresa	Meta (Facebook)
Producto	Private Lift Measurement (PLM)
Inicio	2020, en pruebas con socios seleccionados
Estado	2023: usado por algunos de sus mayores anunciantes
Código	Abierto: `github.com/facebookresearch/fbpcf`
Infraestructura	AWS, contenedores que se levantan por ejecución
Simetría	El anunciante despliega **el mismo** software que Meta
Versión 1.0	Usaba **garbled circuits** (sobre EMP-Toolkit)
Versión 2.0	Cambió a secret sharing (GMW)

Los datos concretos del caso

Empresa	Meta (Facebook)
Producto	Private Lift Measurement (PLM)
Inicio	2020, en pruebas con socios seleccionados
Estado	2023: usado por algunos de sus mayores anunciantes
Código	Abierto: `github.com/facebookresearch/fbpcf`
Infraestructura	AWS, contenedores que se levantan por ejecución
Simetría	El anunciante despliega **el mismo** software que Meta
Versión 1.0	Usaba **garbled circuits** (sobre EMP-Toolkit)
Versión 2.0	Cambió a secret sharing (GMW)

¿Por qué cambiaron? Por lo que vimos hace un momento

Meta reporta que la v2 usa **dos órdenes de magnitud menos tráfico de red** que la versión con garbled circuits. **El cuello de botella era exactamente el que predijimos: los bytes.**

La lección de ese caso

Los garbled circuits llevaron el sistema a producción...
y después fueron reemplazados por su técnica hermana.

La lección de ese caso

Los garbled circuits llevaron el sistema a producción...
y después fueron reemplazados por su técnica hermana.

Lo que esto confirma

- La técnica **funciona en producción real**, no solo en papel.
- Su límite es la **comunicación**, no la seguridad ni la velocidad.
- En sumas y conteos, secret sharing gana.

La lección de ese caso

Los garbled circuits llevaron el sistema a producción...
y después fueron reemplazados por su técnica hermana.

Lo que esto confirma

- La técnica **funciona en producción real**, no solo en papel.
- Su límite es la **comunicación**, no la seguridad ni la velocidad.
- En sumas y conteos, secret sharing gana.

Dónde siguen ganando

- **Comparaciones** (ReLU en redes neuronales, máximos, ordenamientos).
- Redes con **latencia alta**: número constante de rondas.
- Cuando el circuito es **pequeño** pero la lógica es compleja.

La lección de ese caso

Los garbled circuits llevaron el sistema a producción...
y después fueron reemplazados por su técnica hermana.

Lo que esto confirma

- La técnica **funciona en producción real**, no solo en papel.
- Su límite es la **comunicación**, no la seguridad ni la velocidad.
- En sumas y conteos, secret sharing gana.

Dónde siguen ganando

- **Comparaciones** (ReLU en redes neuronales, máximos, ordenamientos).
- Redes con **latencia alta**: número constante de rondas.
- Cuando el circuito es **pequeño** pero la lógica es compleja.

También: Meta y Mozilla desarrollan juntos **IPA** (*Interoperable Private Attribution*), con interés de Google Chrome.

Y la frontera de investigación: inferencia privada

El escenario

Un hospital quiere usar un modelo de diagnóstico que vive en un servidor. El hospital **no puede enviar la radiografía** (datos de paciente) y la empresa **no quiere entregar el modelo** (es su propiedad intelectual). Otra vez: dos partes, dos secretos, un resultado.

Y la frontera de investigación: inferencia privada

El escenario

Un hospital quiere usar un modelo de diagnóstico que vive en un servidor. El hospital **no puede enviar la radiografía** (datos de paciente) y la empresa **no quiere entregar el modelo** (es su propiedad intelectual). Otra vez: dos partes, dos secretos, un resultado.

Medición	**Resultado**	**Fuente**
Una operación de comparación (EMP-Toolkit, 64 bits)	**0.098 ms** (garbler) **0.099 ms** (evaluador)	TruncFormer (2024)

Y la frontera de investigación: inferencia privada

El escenario

Un hospital quiere usar un modelo de diagnóstico que vive en un servidor. El hospital **no puede enviar la radiografía** (datos de paciente) y la empresa **no quiere entregar el modelo** (es su propiedad intelectual). Otra vez: dos partes, dos secretos, un resultado.

Medición	**Resultado**	**Fuente**
Una operación de comparación (EMP-Toolkit, 64 bits)	**0.098 ms** (garbler) **0.099 ms** (evaluador)	TruncFormer (2024)
Un token de Llama-7B privado	$>$**200 segundos**	PUMA

Y la frontera de investigación: inferencia privada

El escenario

Un hospital quiere usar un modelo de diagnóstico que vive en un servidor. El hospital **no puede enviar la radiografía** (datos de paciente) y la empresa **no quiere entregar el modelo** (es su propiedad intelectual). Otra vez: dos partes, dos secretos, un resultado.

Medición	**Resultado**	**Fuente**
Una operación de comparación (EMP-Toolkit, 64 bits)	**0.098 ms** (garbler) **0.099 ms** (evaluador)	TruncFormer (2024)
Un token de Llama-7B privado	>**200 segundos**	PUMA

El muro de hoy

Una operación suelta es rapidísima, pero un modelo de lenguaje necesita *miles de millones*. Por eso se investiga **acelerarlo en hardware** (APINT, KAIST, ICCAD 2024).

Rendimiento real y herramientas

Números concretos

JustGarble (2013) cifra una puerta en **23.2 ciclos de reloj** $\approx$ **7.25 nanosegundos**, midiendo sobre un circuito de AES de 36,500 puertas.

Eso son del orden de **decenas de millones de puertas por segundo** en una laptop normal.

Bibliotecas que puedes usar

- **Obliv-C**: C con la palabra clave `obliv`. Ideal para aprender.
- **EMP-toolkit**: C++, el estándar de investigación.
- **ABY / MOTION**: mezclan garbled circuits con otras técnicas de MPC.
- **MP-SPDZ**: decenas de protocolos para experimentar.

Rendimiento real y herramientas

Números concretos

JustGarble (2013) cifra una puerta en **23.2 ciclos de reloj** $\approx$ **7.25 nanosegundos**, midiendo sobre un circuito de AES de 36,500 puertas.

Eso son del orden de **decenas de millones de puertas por segundo** en una laptop normal.

Bibliotecas que puedes usar

- **Obliv-C**: C con la palabra clave `obliv`. Ideal para aprender.
- **EMP-toolkit**: C++, el estándar de investigación.
- **ABY / MOTION**: mezclan garbled circuits con otras técnicas de MPC.
- **MP-SPDZ**: decenas de protocolos para experimentar.

En Obliv-C, el problema de los millonarios completo cabe en **una línea**:

```
obliv if (x > y)
  resultado = true;
```

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.
4. Alicia envía las tablas y las etiquetas de **sus** bits.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.
4. Alicia envía las tablas y las etiquetas de **sus** bits.
5. Bob obtiene las de **sus** bits por **oblivious transfer**, sin revelarlos.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.
4. Alicia envía las tablas y las etiquetas de **sus** bits.
5. Bob obtiene las de **sus** bits por **oblivious transfer**, sin revelarlos.
6. Bob **evalúa a ciegas**: una fila por puerta, encadenando salidas como entradas.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.
4. Alicia envía las tablas y las etiquetas de **sus** bits.
5. Bob obtiene las de **sus** bits por **oblivious transfer**, sin revelarlos.
6. Bob **evalúa a ciegas**: una fila por puerta, encadenando salidas como entradas.
7. Solo la **etiqueta final** se traduce al resultado.

Resumen del protocolo

1. La función se escribe como **circuito de puertas** (público).
2. Alicia da a cada cable **dos etiquetas aleatorias** de 128 bits.
3. Cada puerta se vuelve una **tabla de 4 filas cifradas con AES** y barajadas.
4. Alicia envía las tablas y las etiquetas de **sus** bits.
5. Bob obtiene las de **sus** bits por **oblivious transfer**, sin revelarlos.
6. Bob **evalúa a ciegas**: una fila por puerta, encadenando salidas como entradas.
7. Solo la **etiqueta final** se traduce al resultado.

Nadie ve los datos del otro. Solo emerge el resultado.

Conclusión

Durante toda la historia, computar juntos exigió
confiar en alguien: un notario, un banco, un servidor.

Conclusión

Durante toda la historia, computar juntos exigió
confiar en alguien: un notario, un banco, un servidor.

Los garbled circuits demuestran que
esa exigencia **era falsa**.

Conclusión

Durante toda la historia, computar juntos exigió
confiar en alguien: un notario, un banco, un servidor.

Los garbled circuits demuestran que
esa exigencia **era falsa**.

La confianza se puede fabricar con matemáticas.

Referencias

Fundamentos

- Yao (1986). *How to Generate and Exchange Secrets*. FOCS.
- Malkhi, Nisan, Pinkas, Sella (2004). *Fairplay*. USENIX Security.
- Beaver, Micali, Rogaway (1990). *The Round Complexity of Secure Protocols*. STOC.

Optimizaciones

- Kolesnikov, Schneider (2008). *Free XOR Gates*. ICALP.
- Bellare, Hoang, Keelveedhi, Rogaway (2013). *Efficient Garbling from a Fixed-Key Blockcipher*. IEEE S&P. (JustGarble)
- Zahur, Rosulek, Evans (2015). *Two Halves Make a Whole*. EUROCRYPT. (half-gates)
- Rosulek, Roy (2021). *Three Halves Make a Whole?*. CRYPTO.

Aplicaciones y datos citados

- Meta. *Private Computation Framework*. `github.com/facebookresearch/fbpcf`
- Meta (2021). *Privacy-Enhancing Technologies and Ads*. `about.fb.com/news/2021/08/`
- Mishra et al. (2020). *DELPHI*.
- TruncFormer (2024). arXiv:2412.01042.
- Cho et al. (2024). *APINT*. ICCAD.
- Bogetoft et al. (2009). *Secure MPC Goes Live*. Financial Cryptography.

Herramientas

- `github.com/emp-toolkit`
- `github.com/samee/obliv-c`
- `github.com/data61/MP-SPDZ`

¿Preguntas?

Gracias por su atención.